

InvestSmart System Proposal

Group V InvestSmart System Proposal

1. Project Title:

InvestSmart: A Financial Management and Investment Recommendation Web Application

2. Project Overview:

InvestSmart is a three-tier financial management web application designed to help users record their income, expenses, savings, and investment preferences, then receive suitable bank investment recommendations. The system includes a public home page, a single login and signup flow, client dashboards, admin dashboards, investment provider management, saved investment plans, reports, audit trails, OTP verification, and MySQL-backed data storage.

The system is designed for a student project environment using HTML, CSS, JavaScript, PHP, and MySQL. The frontend and backend are separated into different files and layers so the project clearly demonstrates a three-tier architecture.

3. Problem Statement:

Many users do not know which bank investment option suits their salary, savings, risk tolerance, liquidity preference, and investment goal. They may compare banks manually and make decisions without structured financial data. InvestSmart solves this by collecting the user's financial information, comparing it against stored bank investment criteria, and recommending the most suitable provider.

4. Aim:

The aim of InvestSmart is to provide a simple, secure, and professional web-based system that helps users manage financial data and receive investment recommendations from stored bank/provider information.

5. Objectives:

- Allow users to register and sign in using one shared login page.
- Verify logins using a demonstration OTP process.
- Allow clients to capture and update their financial profile.
- Calculate net salary automatically from gross salary, deductions, and expenses.
- Recommend banks based on investment amount, risk, liquidity, horizon, monthly contribution, and goal.
- Store recommended and saved investment plans.
- Display user reports with profile, finances, plans, providers, and charts.
- Allow administrators to manage users and bank/provider records.
- Provide admin analytics and audit trail information.
- Use MySQL tables with clear relationships and referential integrity.

6. Scope of the System:

Included:

- Home page
- Signup and login with OTP
- Client dashboard
- Profile management
- My Finances
- Investment Calculator
- My Plans
- My Report
- Admin dashboard
- Manage Users
- Manage Providers/Banks
- Admin Reports and analytics
- Audit trail
- Forgot password with on-screen OTP for school demonstration

InvestSmart System Proposal

Excluded for this version:

- Real SMS/email OTP delivery
- Online payment processing
- Live bank API integration
- Production hosting of the backend on a cloud server

7. Target Users:

- Clients: normal users who want to manage finances and receive investment recommendations.
- Administrators: system users who manage clients, banks/providers, reports, and audit information.
- Lecturers/assessors: evaluators who inspect the system design, implementation, database, and test evidence.

8. Three-Tier Architecture:

Presentation Layer:

The frontend is built with HTML, CSS, and JavaScript. It handles the home page, forms, dashboards, tables, charts, and user interaction.

Application Layer:

The backend is built with PHP. It handles validation, authentication, OTP generation, recommendation logic, CRUD operations, role checks, and API responses.

Data Layer:

The database is MySQL. It stores users, clients, admins, financial profiles, banks, investment plans, audit logs, and OTP records.

9. Main Functional Requirements:

- The system shall allow a new client to create an account.
- The system shall allow both clients and admins to log in from the same login page.
- The system shall request OTP verification after correct email and password.
- The system shall route clients to the client dashboard and admins to the admin dashboard.
- The system shall calculate net salary from salary inputs.
- The system shall recommend investment banks using stored criteria.
- The system shall store the selected bank website link in saved plans.
- The system shall allow clients to view, edit, and delete saved plans.
- The system shall allow administrators to manage bank/provider records.
- The system shall allow administrators to activate/deactivate users.
- The system shall record important actions in an audit trail.

10. Non-Functional Requirements:

- Usability: the interface must be clean, responsive, and professional.
- Security: passwords must be hashed and protected routes must require authentication.
- Maintainability: frontend, backend, and database scripts must be separated.
- Reliability: database foreign keys must preserve relationship integrity.
- Performance: pages should load quickly and API responses should return JSON.
- Portability: the backend should run on XAMPP and the frontend can be hosted separately.

11. Fully Attributed ERD / Data Dictionary:

Table: users

Attributes: id INT PK AUTO_INCREMENT; email VARCHAR(180) UNIQUE NOT NULL; password_hash VARCHAR(255) NOT NULL; role ENUM(client, admin) DEFAULT client; status ENUM(active, inactive) DEFAULT active; created_at TIMESTAMP.

Purpose: stores common login credentials and account state for all system users.

Table: clients

Attributes: id INT PK AUTO_INCREMENT; user_id INT FK UNIQUE; full_name VARCHAR(120); surname

InvestSmart System Proposal

VARCHAR(120); id_number VARCHAR(20) UNIQUE; contact_info VARCHAR(180); created_at TIMESTAMP.

Purpose: stores normal user/client identity details linked to users.

Table: admins

Attributes: id INT PK AUTO_INCREMENT; user_id INT FK UNIQUE; full_name VARCHAR(120); surname VARCHAR(120); employee_code VARCHAR(40) UNIQUE; contact_info VARCHAR(180); created_at TIMESTAMP.

Purpose: stores administrator identity details linked to users.

Table: financial_profiles

Attributes: id INT PK AUTO_INCREMENT; client_id INT FK UNIQUE; gross_salary DECIMAL(12,2); deductions DECIMAL(12,2); monthly_expenses DECIMAL(12,2); current_savings DECIMAL(12,2); net_salary DECIMAL(12,2); updated_at TIMESTAMP.

Purpose: stores each client's financial profile.

Table: banks

Attributes: id INT PK AUTO_INCREMENT; name VARCHAR(160); contact_info VARCHAR(220); website VARCHAR(255); plan_type VARCHAR(80); expected_return DECIMAL(5,2); risk ENUM(Low, Medium, High); liquidity ENUM(High, Medium, Low); horizon ENUM(Short, Medium, Long); allows_monthly TINYINT(1); details TEXT; created_by_admin_id INT FK NULL; created_at TIMESTAMP.

Purpose: stores bank/provider investment options and recommendation criteria.

Table: investment_plans

Attributes: id INT PK AUTO_INCREMENT; client_id INT FK; bank_id INT FK NULL; user_plan_name VARCHAR(160); investment_amount DECIMAL(12,2); monthly_contribution TINYINT(1); monthly_amount DECIMAL(12,2); investment_goal VARCHAR(255); risk ENUM(Low, Medium, High); liquidity ENUM(High, Medium, Low); horizon ENUM(Short, Medium, Long); expected_return DECIMAL(5,2); score INT; bank_investment_url VARCHAR(255); created_at TIMESTAMP; updated_at TIMESTAMP.

Purpose: stores saved recommendations and client investment plans.

Table: activity_logs

Attributes: id INT PK AUTO_INCREMENT; user_id INT FK NULL; client_id INT FK NULL; admin_id INT FK NULL; action VARCHAR(80); entity_type VARCHAR(80); entity_id INT NULL; description VARCHAR(255); created_at TIMESTAMP.

Purpose: stores audit trail events for clients and administrators.

Table: auth_otps

Attributes: id INT PK AUTO_INCREMENT; user_id INT FK; purpose ENUM(login, password_reset); otp_hash VARCHAR(255); attempts INT; expires_at DATETIME; used_at DATETIME NULL; created_at TIMESTAMP; index idx_auth_otps_user_purpose(user_id, purpose, expires_at).

Purpose: stores hashed OTP records for login and password reset.

12. ERD Relationships:

- users 1 to 0..1 clients: one user account can have one client profile.
- users 1 to 0..1 admins: one user account can have one admin profile.
- clients 1 to 0..1 financial_profiles: one client can have one financial profile.
- clients 1 to many investment_plans: one client can save many plans.
- banks 1 to many investment_plans: one bank can be recommended in many saved plans.
- admins 1 to many banks: one admin can create or maintain many bank records.
- users 1 to many auth_otps: one user can receive many OTP records over time.
- users/clients/admins 1 to many activity_logs: actions are recorded for audit and accountability.

13. Recommendation Logic Summary:

The calculator compares the user's selected risk, liquidity, horizon, monthly contribution preference, and investment amount against the bank/provider records. Matching criteria increase a recommendation score. The provider with the best score and suitable investment

InvestSmart System Proposal

attributes is displayed to the user with expected return, contact details, and investment website link.

14. Security Summary:

Passwords are hashed using PHP password_hash. Login requires email, password, and OTP verification. The OTP is shown on screen for school demonstration, but it is stored as a hash in the database. Protected backend actions require authentication tokens, and admin-only actions require role checking.

15. Expected Deliverables:

- Frontend source code
- PHP backend source code
- MySQL database script
- ERD diagram and data dictionary
- Installation/deployment guide
- Test cases document
- Security information document
- Working demonstration using XAMPP, hosted frontend, and ngrok when required

16. Conclusion:

InvestSmart provides a complete student-level three-tier web application that combines financial profile management, investment recommendation, role-based administration, secure login, OTP demonstration, reporting, and database-driven decision support.